

Spark Data Projects: Healthcare, E-commerce and Supply Chain

Overview and environment

This document describes three PySpark scripts that profile and analyze three datasets stored in different formats (Avro, Parquet and CSV) on a Hadoop cluster. Each section shows the script's code step by step, followed by a screenshot of its output and a short reading of the results.

Script	Dataset	Format	What it does
HealthcareScript.py	Hospital admissions	Avro	Quality checks, saves a Parquet table, runs SQL queries
EcommerceScript.py	Shopping events	Parquet	Quality checks, sales funnel, revenue, cart abandonment
SupplyChainScript.py	Supply chain orders	CSV	Quality checks, delivery performance, sales trends

Environment

Component	Value
Apache Spark	4.1.3 (Scala 2.13), run locally with <code>local[*]</code>
Python	3.12
Java	OpenJDK 21
Storage	HDFS on the <code>hadoop-master</code> machine
Avro support	<code>spark-avro_2.13-4.1.3.jar</code> in Spark's jars folder

Each script is run from the terminal with `spark-submit`. Adding `2>/dev/null` hides Spark's log messages so only the script's own output is shown.

```
spark-submit HealthcareScript.py 2>/dev/null
```

All three scripts follow the same pattern: load the data, inspect its schema and a sample, check for missing values, check other quality problems, and then run analysis queries.

Script 1: Healthcare (Avro to Parquet table)

The script turns a 55,500-row Avro file into a de-duplicated Parquet table of 54,966 rows and answers eight analytical questions with SQL.

Item	Value
Input	/HealthcareData/healthcare_dataset.avro (Avro, 55,500 rows, 15 columns)
Output table	healthcare.admissions (Parquet, 54,966 rows)
Run command	spark-submit HealthcareScript.py

1. Setup and data loading

The script starts a Spark session with Hive support, so the table definition survives after the script ends, then reads the Avro file from HDFS. Reading Avro needs the `spark-avro` jar in Spark's jars folder.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (SparkSession.builder
        .appName("HealthcareProject")
        .enableHiveSupport()
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

# data loading
df = spark.read.format("avro").load("/HealthcareData/healthcare_dataset.avro")
```

2. Dataset size and schema

`count()` gives the number of rows and `len(df.columns)` the number of columns. `printSchema()` shows each column's name and data type.

```
total = df.count()
print("=====DATASET SIZE=====")
print("rows:", total, "| columns:", len(df.columns))

print("=====SCHEMA=====")
df.printSchema()
```

```
hadoop@hadoop-master:~$ spark-submit HealthcareScript.py 2>/dev/null
=====DATASET SIZE=====
rows: 55500 | columns: 15
=====SCHEMA=====
root
 |-- Name: string (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Gender: string (nullable = true)
 |-- Blood_Type: string (nullable = true)
 |-- Medical_Condition: string (nullable = true)
 |-- Date_of_Admission: date (nullable = true)
 |-- Doctor: string (nullable = true)
 |-- Hospital: string (nullable = true)
 |-- Insurance_Provider: string (nullable = true)
 |-- Billing_Amount: double (nullable = true)
 |-- Room_Number: integer (nullable = true)
 |-- Admission_Type: string (nullable = true)
 |-- Discharge_Date: date (nullable = true)
 |-- Medication: string (nullable = true)
 |-- Test_Results: string (nullable = true)
```

The file has 55,500 rows and 15 columns. The two date columns are read as `date`, and `Billing_Amount` as `double`.

3. Sample data

```
print("=====SAMPLE DATA (first 5 rows)=====")
df.show(5)
```

```
=====SAMPLE DATA (first 5 rows)=====
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Name|Age|Gender|Blood_Type|Medical_Condition|Date_of_Admission| Doctor| Hospital|Insurance_Provider| Billing_Amount|Room_Number|Admission_Type|Discharge_Date| Medication|Test_Result|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Bobby JacksOn| 30| Male| B-| Cancer| 2024-01-31| Matthew Smith| Sons and Miller| Blue Cross|18856.2813059782| 328| Urgent| 2024-02-02|Paracetamol| Norma|
| Leslie TErRy| 62| Male| A+| Obesity| 2019-08-20| Samantha Davies| Kin Inc| Medicare|33643.3272865779| 265| Emergency| 2019-08-26| Ibuprofen|Inconclusiv|
| DaNnY sHtH| 76|Female| A-| Obesity| 2022-09-22|Tiffany Mitchell| Cook PLC| Aetna|27955.0968788425| 205| Emergency| 2022-10-07| Aspirin| Norma|
| andrEw waTtS| 28|Female| O+| Diabetes| 2020-11-18| Kevin Wells|Hernandez Rogers ...| Medicare|37909.7824098753| 450| Elective| 2020-12-18| Ibuprofen| Abnorma|
| IdriEWNE bEl| 43|Female| AB+| Cancer| 2022-09-19| Kathleen Hanna| White-White| Aetna|14238.3178139376| 450| Urgent| 2022-10-09| Penicillin| Abnorma|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

The sample already shows a text problem: names have mixed casing, such as `Bobby JacksOn` and `andrEw waTtS`.

4. Null values per column

For each column, the code turns every NULL into 1 and every other value into 0, then adds them up. The list comprehension repeats this for all 15 columns in a single pass over the data.

```
print("=====NULL VALUES PER COLUMN=====")
df.select([F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c) for c in
df.columns]).show(vertical=True)
```

```
=====NULL VALUES PER COLUMN=====
-RECORD 0-----
| Name| 0|
| Age| 0|
| Gender| 0|
| Blood_Type| 0|
| Medical_Condition| 0|
| Date_of_Admission| 0|
| Doctor| 0|
| Hospital| 0|
| Insurance_Provider| 0|
| Billing_Amount| 0|
| Room_Number| 0|
| Admission_Type| 0|
| Discharge_Date| 0|
| Medication| 0|
| Test_Results| 0|
```

Every column has 0 NULLs, so no missing-value handling is needed.

5. Duplicate rows and statistics

`dropDuplicates()` keeps one copy of each identical row. Subtracting the new row count from the original gives the number of duplicates. The statistics summary is calculated on the de-duplicated data.

```
print("=====DUPLICATE ROWS=====")
cleaned_df = df.dropDuplicates()
print("number of duplicate rows:", total - cleaned_df.count())

# statistic summary
print("=====STATISTICS SUMMARY (after removing duplicates)=====")
cleaned_df.describe().show()
```

```

=====DUPLICATE ROWS=====
number of duplicate rows: 534
=====STATISTICS SUMMARY (after removing duplicates)=====
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|summary| Name| Age|Gender|Blood_Type|Medical_Condition| Doctor| Hospital|Insurance_Provider| Billing_Amount| Room_Number|Admission_Type|Medication|Test_Results|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| count| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966| 54966|
| mean| NULL|51.53518538733835| NULL| NULL| NULL| NULL| NULL| NULL|25544.38628438365| 381.1244841771277| NULL| NULL| NULL|
| stddev| NULL|19.68566897257976| NULL| NULL| NULL| NULL| NULL| NULL|14288.489718716349| 115.22314386428152| NULL| NULL| NULL|
| min|AARON Duncan| 13|Female| A+| Arthritis|Aaron Acevedo| Abbott Inc| Aetna| -2008.4921398591| 101| Elective| Aspirin| Abnormal|
| max|zachary WALL| 89| Male| O-| Obesity| Zoe Wallace|and Zuniga Thomps...| UnitedHealthcare| 52764.2767364692| 500| Urgent|Penicillin| Normal|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

The data contains 534 duplicate rows, which leaves 54,966 rows. Age runs from 13 to 89 and Room_Number from 101 to 500, both realistic. The minimum Billing_Amount is -2008.49, so some bills are invalid.

6. Saving the table

The cleaned DataFrame is written as Parquet and registered as the table `healthcare.admissions`. Parquet stores data by column, which makes analytical queries faster than Avro. The source Avro file is not changed.

```

print("=====SAVING AS TABLE=====")
spark.sql("CREATE DATABASE IF NOT EXISTS healthcare")

(cleaned_df.write
 .mode("overwrite")
 .format("parquet")
 .saveAsTable("healthcare.admissions"))

print("=====TABLES IN HEALTHCARE DATABASE=====")
spark.sql("SHOW TABLES IN healthcare").show()

```

```

=====SAVING AS TABLE=====
=====TABLES IN HEALTHCARE DATABASE=====
+-----+-----+-----+
| namespace| tableName|isTemporary|
+-----+-----+-----+
| healthcare|admissions| false|
+-----+-----+-----+

```

The table `admissions` exists in the `healthcare` database, and `isTemporary` is false, so it is a saved table and not a temporary view.

```

print("=====ROW COUNT CHECK=====")
spark.sql("SELECT COUNT(*) AS total_rows FROM healthcare.admissions").show()

```

```

=====ROW COUNT CHECK=====
+-----+
| total_rows|
+-----+
|      54966|
+-----+

```

The table holds 54,966 rows, matching the de-duplicated DataFrame.

7. Analytical view

The queries run on a temporary view that removes rows with a billing amount of 0 or less and rows where the discharge date is before the admission date. It also adds Length_of_Stay in days. The view reduces the table from 54,966 to 54,860 rows, and the saved table itself is not changed.

```
spark.sql("""
CREATE OR REPLACE TEMP VIEW valid_admissions AS
SELECT *, DATEDIFF(Discharge_Date, Date_of_Admission) AS Length_of_Stay
FROM healthcare.admissions
WHERE Billing_Amount > 0
      AND Discharge_Date >= Date_of_Admission
""")
```

8. Query: overall KPIs

```
print("=====OVERALL KPIs=====")
spark.sql("""
SELECT COUNT(*) AS total_admissions,
       ROUND(SUM(Billing_Amount), 0) AS total_revenue,
       ROUND(AVG(Billing_Amount), 2) AS avg_bill,
       ROUND(AVG(Length_of_Stay), 1) AS avg_stay_days
FROM valid_admissions
""").show()
```

```
=====OVERALL KPIs=====
+-----+-----+-----+-----+
|total_admissions|total_revenue|avg_bill|avg_stay_days|
+-----+-----+-----+-----+
|          54860|1.404121601E9|25594.63|          15.5|
+-----+-----+-----+-----+
```

There are 54,860 valid admissions with total revenue of about 1.404 billion. The average bill is 25,594.63 and the average stay is 15.5 days.

9. Query: admissions, average bill and stay per medical condition

```
print("=====ADMISSIONS, AVG BILL AND AVG STAY PER MEDICAL CONDITION=====")
spark.sql("""
SELECT Medical_Condition,
       COUNT(*) AS admissions,
       ROUND(AVG(Billing_Amount), 2) AS avg_bill,
       ROUND(AVG(Length_of_Stay), 1) AS avg_stay_days
FROM valid_admissions
GROUP BY Medical_Condition
ORDER BY avg_bill DESC
""").show()
```

```

=====ADMISSIONS, AVG BILL AND AVG STAY PER MEDICAL CONDITION=====
+-----+-----+-----+-----+
|Medical_Condition|admissions|avg_bill|avg_stay_days|
+-----+-----+-----+-----+
|      Obesity|      9127|25859.22|      15.4|
|      Diabetes|      9197|25714.33|      15.4|
|      Asthma|      9077|25685.39|      15.7|
|Hypertension|      9131|25559.84|      15.4|
|      Arthritis|      9207| 25542.9|      15.5|
|      Cancer|      9121|25205.92|      15.5|
+-----+-----+-----+-----+

```

Obesity has the highest average bill (25,859.22) and Cancer the lowest (25,205.92). Average stays are between 15.4 and 15.7 days for all six conditions.

10. Query: revenue per insurance provider

```

print("=====REVENUE PER INSURANCE PROVIDER=====")
spark.sql("""
    SELECT Insurance_Provider,
           COUNT(*) AS admissions,
           ROUND(SUM(Billing_Amount), 0) AS total_revenue,
           ROUND(AVG(Billing_Amount), 2) AS avg_bill
    FROM valid_admissions
    GROUP BY Insurance_Provider
    ORDER BY total_revenue DESC
""").show()

```

```

=====REVENUE PER INSURANCE PROVIDER=====
+-----+-----+-----+-----+
|Insurance_Provider|admissions|total_revenue|avg_bill|
+-----+-----+-----+-----+
|      Cigna|      11115| 2.84346461E8|25582.23|
|      Medicare|      11018| 2.82921172E8|25678.09|
|      Blue Cross|      10937| 2.80416694E8|25639.27|
|UnitedHealthcare|      10995| 2.79920532E8|25458.89|
|      Aetna|      10795| 2.76516742E8|25615.26|
+-----+-----+-----+-----+

```

Cigna brings the most revenue (about 284.3 million from 11,115 admissions) and Aetna the least (about 276.5 million). Average bills are close, from 25,458.89 to 25,678.09.

11. Query: admission type

```

print("=====ADMISSION TYPE: COUNT, AVG BILL, AVG STAY=====")
spark.sql("""
    SELECT Admission_Type,
           COUNT(*) AS admissions,
           ROUND(AVG(Billing_Amount), 2) AS avg_bill,
           ROUND(AVG(Length_of_Stay), 1) AS avg_stay_days
    FROM valid_admissions
    GROUP BY Admission_Type
    ORDER BY admissions DESC
""").show()

```

```

=====ADMISSION TYPE: COUNT, AVG BILL, AVG STAY=====
+-----+-----+-----+-----+
|Admission_Type|admissions|avg_bill|avg_stay_days|
+-----+-----+-----+-----+
|      Elective|      18437|25663.34|      15.5|
|        Urgent|      18353|25568.44|      15.4|
|      Emergency|      18070|25551.13|      15.6|
+-----+-----+-----+-----+

```

Elective (18,437), Urgent (18,353) and Emergency (18,070) admissions are almost evenly split, with similar average bills and stays.

12. Query: age group analysis

A CASE WHEN expression assigns each patient to an age group before grouping.

```

print("=====AGE GROUP ANALYSIS=====")
spark.sql("""
    SELECT CASE WHEN Age < 18 THEN '0-17'
               WHEN Age < 40 THEN '18-39'
               WHEN Age < 65 THEN '40-64'
               ELSE '65+' END AS age_group,
           COUNT(*) AS admissions,
           ROUND(AVG(Billing_Amount), 2) AS avg_bill,
           ROUND(AVG(Length_of_Stay), 1) AS avg_stay_days
    FROM valid_admissions
    GROUP BY 1
    ORDER BY 1
""").show()

```

```

=====AGE GROUP ANALYSIS=====
+-----+-----+-----+-----+
|age_group|admissions|avg_bill|avg_stay_days|
+-----+-----+-----+-----+
|    0-17|        116|28512.86|      15.3|
|   18-39|       17561| 25656.6|      15.4|
|   40-64|      20294|25615.18|      15.5|
|    65+|       16889|25485.47|      15.6|
+-----+-----+-----+-----+

```

The 40-64 group is the largest with 20,294 admissions. The 0-17 group has only 116 admissions, so its higher average bill (28,512.86) rests on few rows.

13. Query: yearly trend

```

print("=====YEARLY TREND=====")
spark.sql("""
    SELECT YEAR(Date_of_Admission) AS year,
           COUNT(*) AS admissions,
           ROUND(SUM(Billing_Amount), 0) AS total_revenue
    FROM valid_admissions

```

```
GROUP BY 1
ORDER BY 1
""").show()
```

```
=====YEARLY TREND=====
+----+-----+-----+
|year|admissions|total_revenue|
+----+-----+-----+
|2019|      7280| 1.87518405E8|
|2020|     11157| 2.83959614E8|
|2021|     10795| 2.77186578E8|
|2022|     10891| 2.78625585E8|
|2023|     10918| 2.79680954E8|
|2024|      3819| 9.7150467E7|
+----+-----+-----+
```

Admissions are steady at roughly 10,800 to 11,200 a year from 2020 to 2023. The lower counts in 2019 (7,280) and 2024 (3,819) probably mean the data covers only part of those two years.

14. Query: abnormal test results per medication

```
print("=====PERCENTAGE OF ABNORMAL TEST RESULTS PER MEDICATION=====")
spark.sql("""
    SELECT Medication,
           COUNT(*) AS admissions,
           ROUND(100 * SUM(CASE WHEN Test_Results = 'Abnormal' THEN 1 ELSE 0 END) /
COUNT(*), 1) AS pct_abnormal
    FROM valid_admissions
    GROUP BY Medication
    ORDER BY pct_abnormal DESC
""").show()
```

```
=====PERCENTAGE OF ABNORMAL TEST RESULTS PER MEDICATION=====
+-----+-----+-----+
| Medication|admissions|pct_abnormal|
+-----+-----+-----+
|Paracetamol|     10945|        33.7|
|   Aspirin|     10963|        33.7|
|Ibuprofen|     11002|        33.6|
|Penicillin|     10932|        33.4|
|   Lipitor|     11018|        33.2|
+-----+-----+-----+
```

The share of abnormal results is almost identical across the five medications, from 33.2% (Lipitor) to 33.7% (Paracetamol and Aspirin).

15. Query: top 3 conditions by revenue in each age group

`RANK()` is a window function. It numbers the conditions inside each age group by revenue, and the outer query keeps ranks 1 to 3.

```
print("=====TOP 3 CONDITIONS BY REVENUE IN EACH AGE GROUP=====")
spark.sql("""
    SELECT * FROM (
        SELECT CASE WHEN Age < 40 THEN 'under 40'
                   WHEN Age < 65 THEN '40-64'
                   ELSE '65+' END AS age_group,
               Medical_Condition,
               ROUND(SUM(Billing_Amount), 0) AS revenue,
               RANK() OVER (PARTITION BY CASE WHEN Age < 40 THEN 'under 40'
                                           WHEN Age < 65 THEN '40-64'
                                           ELSE '65+' END
                           ORDER BY SUM(Billing_Amount) DESC) AS rnk
        FROM valid_admissions
        GROUP BY 1, 2
    )
    WHERE rnk <= 3
    ORDER BY age_group, rnk
""").show()
```

```
=====TOP 3 CONDITIONS BY REVENUE IN EACH AGE GROUP=====
+-----+-----+-----+-----+
|age_group|Medical_Condition|   revenue|rnk|
+-----+-----+-----+-----+
|   40-64|         Obesity|8.9926684E7|  1|
|   40-64|        Diabetes|8.8817219E7|  2|
|   40-64|    Hypertension|8.6684681E7|  3|
|    65+|          Asthma|7.3507566E7|  1|
|    65+|        Arthritis|7.3279274E7|  2|
|    65+|        Diabetes|7.2639066E7|  3|
| under 40|         Obesity|7.7002699E7|  1|
| under 40|          Asthma|7.6712445E7|  2|
| under 40|        Arthritis|7.5558431E7|  3|
+-----+-----+-----+-----+
```

Obesity ranks first in the 40-64 group (about 89.9 million) and in the under-40 group (about 77.0 million). Asthma ranks first for patients aged 65 and over (about 73.5 million).

Script 2: E-commerce (Parquet)

The script profiles a Parquet file of about 107.9 million shopping events. It checks data quality and then measures the sales funnel, revenue, cart abandonment and activity by hour.

Item	Value
Input	/EcommerceData/ecommerce_final.parquet (Parquet, about 107.9 million rows, 9 columns)
Output	Results printed to the console (no table is saved)
Run command	spark-submit EcommerceScript.py

1. Setup and data loading

The `event_time` column is stored with nanosecond precision, which Spark cannot read as a timestamp by default. The setting `nanosAsLong` makes Spark read it as a plain number instead.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("EcommerceProject").getOrCreate()

spark.conf.set("spark.sql.legacy.parquet.nanosAsLong", "true")
df = spark.read.parquet("/EcommerceData/ecommerce_final.parquet")
```

2. Schema

```
print("Schema of the dataset:")
df.printSchema()
```

```
Schema of the dataset:
root
 |-- event_time: long (nullable = true)
 |-- event_type: string (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- category_id: long (nullable = true)
 |-- category_code: string (nullable = true)
 |-- brand: string (nullable = true)
 |-- price: float (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- user_session: integer (nullable = true)
```

The dataset has 9 columns: `event_time` (long), `event_type`, `category_code` and `brand` (string), `price` (float), and four ID columns (integer or long).

3. Sample data

With `vertical=True`, each row is printed as a list of column and value pairs, which is easier to read for a table with many columns. `truncate=False` shows full values.

```
print("Sample of 5 rows:")
df.show(5, truncate=False, vertical=True)
```

```

Sample of 5 rows:
-RECORD 0-----
event_time | 1569888000000000000
event_type | view
product_id | 44600062
category_id | 2103807459595387724
category_code | Unknown
brand | shiseido
price | 35.79
user_id | 541312140
user_session | 10327253
-RECORD 1-----
event_time | 1569888000000000000
event_type | view
product_id | 3900821
category_id | 2053013552326770905
category_code | appliances.environment.water_heater
brand | aqua
price | 33.2
user_id | 554748717
user_session | 13239002
-RECORD 2-----
event_time | 1569888001000000000
event_type | view
product_id | 17200506
category_id | 2053013559792632471
category_code | furniture.living_room.sofa
brand | Unknown
price | 543.1
user_id | 510107250
user_session | 7768391
-RECORD 3-----
event_time | 1569888001000000000
event_type | view
product_id | 1307067
category_id | 2053013558920217191
category_code | computers.notebook
brand | lenovo
price | 251.74
user_id | 550050854
user_session | 11202665
-RECORD 4-----
event_time | 1569888004000000000
event_type | view
product_id | 1004237
category_id | 2053013555631882655
category_code | electronics.smartphone
brand | apple
price | 1081.98
user_id | 535871217
user_session | 17868472
only showing top 5 rows

```

All five rows are view events. `event_time` is a raw nanosecond number, and `brand` or `category_code` can hold the text `Unknown`.

4. Unknown values and null values

Missing values in this dataset were already replaced by the text `Unknown`, so a plain null check would hide them. The script counts `Unknown` values separately. In the null check, `F.when(...)` returns the column name only when the value is `NULL`, and `F.count()` ignores rows where `when` returns nothing, so it counts only the `NULL`s.

```

print("Checking for unknown brand and category_code values...")
no_unknown_brands = df.filter(F.col("brand") == "Unknown").count()
no_unknown_cat_code = df.filter(F.col("category_code") == "Unknown").count()
total = df.count()
print("the number of unknown brands: ", no_unknown_brands, "with percentage ",
      no_unknown_brands/total*100, "%")
print("the number of unknown category codes: ", no_unknown_cat_code, "with percentage ",
      no_unknown_cat_code/total*100, "%")

print("Checking for null values across all columns...")
df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).show()

```

```

Checking for unknown brand and category_code values...
the number of unknown brands: 14731233 with percentage 13.657941174932622 %
the number of unknown category codes: 34608556 with percentage 32.0870372491808 %
Checking for null values across all columns...
+-----+-----+-----+-----+-----+-----+-----+-----+
|event_time|event_type|product_id|category_id|category_code|brand|price|user_id|user_session|
+-----+-----+-----+-----+-----+-----+-----+-----+
|          0|          0|          0|          0|          0|    0|    0|    0|          0|
+-----+-----+-----+-----+-----+-----+-----+-----+

```

About 13.66% of rows (14,731,233) have an unknown brand and 32.09% (34,608,556) have an unknown category code. There are no NULLs in any column.

5. Price check, approximate duplicates and event funnel

Counting exact duplicates across more than 100 million rows would be slow. Instead, the script joins all columns of each row into one text value and estimates how many distinct values exist with `approx_count_distinct`, which uses the HyperLogLog++ algorithm and needs little memory. Total rows minus the distinct estimate gives the approximate number of duplicates. The funnel counts how many events of each type exist.

```

print("Checking for rows with price <= 0...")
zero_or_negative_price = df.filter(F.col("price") <= 0).count()
print("rows with price <= 0:", zero_or_negative_price)

print("Estimating approximate duplicate rows (HyperLogLog++)...")
approx_distinct = df.select(
    F.approx_count_distinct(F.concat_ws("||", *df.columns)).alias("approx_distinct")
).collect()[0]["approx_distinct"]
approx_duplicates = total - approx_distinct
print("approx distinct rows:", approx_distinct)
print("approx duplicate rows:", approx_duplicates)

print("Event type breakdown (funnel counts)...")
df.groupBy("event_type").count().show()

```

```

Checking for rows with price <= 0...
rows with price <= 0: 0
Estimating approximate duplicate rows (HyperLogLog++)...
approx distinct rows: 97057502
approx duplicate rows: 10800870
Event type breakdown (funnel counts)...
+-----+-----+
|event_type|    count|
+-----+-----+
| purchase| 1645281|
|   view|102416114|
|   cart| 3796977|
+-----+-----+

```


No row has a price of 0 or less. About 97.06 million rows are distinct, so roughly 10.80 million rows (about 10%) are estimated duplicates. The funnel shows 102,416,114 views, 3,796,977 cart events and 1,645,281 purchases.

6. Total revenue and average purchase price

Only purchase events count as revenue, so the script filters them before summing the price.

```
print("Calculating total revenue and average purchase price...")
df.filter(F.col("event_type") == "purchase").agg(
    F.sum("price").alias("total_revenue"),
    F.avg("price").alias("avg_purchase_price"),
    F.count("*").alias("num_purchases")
).show()
```

```
Calculating total revenue and average purchase price...
+-----+-----+-----+
|      total_revenue | avg_purchase_price | num_purchases |
+-----+-----+-----+
| 5.009773372876585E8 | 304.49348001202134 |      1645281 |
+-----+-----+-----+
```

The 1,645,281 purchases bring total revenue of about 500.98 million, with an average purchase price of 304.49.

7. Cart abandonment and revenue per category

The abandonment rate is calculated per session. For each `user_session`, the script counts the cart events and the purchase events. A session is abandoned when it has at least one cart event and no purchase. The rate is abandoned sessions divided by all sessions that used the cart.

```
print("Calculating cart abandonment rate...")
session_summary = df.groupBy("user_session").agg(
    F.sum(F.when(F.col("event_type") == "cart", 1).otherwise(0)).alias("cart_adds"),
    F.sum(F.when(F.col("event_type") == "purchase",
1).otherwise(0)).alias("purchases")
)
abandoned = session_summary.filter((F.col("cart_adds") > 0) & (F.col("purchases") ==
0)).count()
total_cart_sessions = session_summary.filter(F.col("cart_adds") > 0).count()
print("cart abandonment rate:", abandoned / total_cart_sessions * 100, "%")
```

The next part ranks the ten categories with the highest purchase revenue.

```
print("Top 10 categories by purchase revenue...")
df.filter(F.col("event_type") == "purchase") \
  .groupBy("category_code") \
  .agg(F.sum("price").alias("revenue"), F.count("*").alias("purchases")) \
  .orderBy(F.desc("revenue")) \
  .show(10, truncate=False)
```

Calculating cart abandonment rate...

cart abandonment rate: 59.39020620775193 %

Top 10 categories by purchase revenue...

category_code	revenue	purchases
electronics.smartphone	3.324380003650665E8	714769
Unknown	5.226486476931971E7	403906
electronics.video.tv	2.067222300884247E7	51360
computers.notebook	1.9502924513010025E7	33807
electronics.clocks	1.1249489293243408E7	40619
appliances.kitchen.washer	1.0410081619476318E7	35753
electronics.audio.headphone	9156344.09686029	70928
appliances.kitchen.refrigerators	8492258.39567566	24117
appliances.environment.vacuum	4440955.586326122	30413
electronics.tablet	3110483.912940979	11640

only showing top 10 rows

About 59.39% of the sessions that added an item to the cart ended without a purchase. Smartphones bring the most revenue (about 332.4 million from 714,769 purchases), roughly two thirds of the total. The Unknown category is second with about 52.3 million, which shows how much the missing category codes affect this ranking.

8. Events by hour of day

`event_time` holds nanoseconds since 1970. Dividing by 1,000,000,000 converts it to seconds, and casting to `timestamp` lets Spark extract the hour. The session time zone is set to UTC so the hours match the data and not the server's local time. A day-of-week column is also prepared for later analysis.

```
print("Event counts by hour of day, ordered busiest to quietest...")
spark.conf.set("spark.sql.session.timeZone", "UTC")

df_time = (df
  .withColumn("event_ts", (F.col("event_time") / 1000000000).cast("timestamp"))
  .withColumn("event_hour", F.hour("event_ts")))
```

```
.withColumn("event_dow", F.dayofweek("event_ts"))
df_time.groupBy("event_hour").count().orderBy(F.desc("count")).show(24)
```

Event counts by hour of day, ordered busiest to quietest...

event_hour	count
16	7419929
15	7296276
17	7002533
14	6889829
13	6154787
8	5970690
9	5898425
10	5811709
7	5811624
6	5715174
11	5670776
12	5623744
5	5468582
18	5276207
4	4887781
3	3877404
19	3635572
2	2692898
20	2125116
1	1373874
21	1232764
0	739146
22	734485
23	549047

Activity peaks at 16:00 UTC (7,419,929 events), followed by 15:00 and 17:00. The hours from 13:00 to 17:00 each have more than 6.1 million events. The quietest hour is 23:00 with 549,047 events.

Script 3: Supply Chain (CSV)

The script profiles a supply chain orders file of 180,519 rows and 53 columns. It checks data quality and then measures delivery performance, sales per category and the monthly sales trend.

Item	Value
Input	/data/DataCoSupplyChainDataset.csv (CSV, 180,519 rows, 53 columns)
Output	Results printed to the console (no table is saved)
Run command	spark-submit SupplyChainScript.py

1. Setup, loading and schema

A CSV file has no stored data types. `header=True` uses the first line as column names, and `inferSchema=True` makes Spark scan the data to guess each column's type. This is slower, but it gives numeric columns instead of plain text.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("SupplyChainProject").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

df = spark.read.csv("/data/DataCoSupplyChainDataset.csv", header=True,
inferSchema=True)
df.printSchema()
```

```
hadoop@hadoop-master:~$ spark-submit SupplyChainScript.py 2>/dev/null
```

```
root
```

```
|-- Type: string (nullable = true)
|-- Days for shipping (real): integer (nullable = true)
|-- Days for shipment (scheduled): integer (nullable = true)
|-- Benefit per order: double (nullable = true)
|-- Sales per customer: double (nullable = true)
|-- Delivery Status: string (nullable = true)
|-- Late_delivery_risk: integer (nullable = true)
|-- Category Id: integer (nullable = true)
|-- Category Name: string (nullable = true)
|-- Customer City: string (nullable = true)
|-- Customer Country: string (nullable = true)
|-- Customer Email: string (nullable = true)
|-- Customer Fname: string (nullable = true)
|-- Customer Id: integer (nullable = true)
|-- Customer Lname: string (nullable = true)
|-- Customer Password: string (nullable = true)
|-- Customer Segment: string (nullable = true)
|-- Customer State: string (nullable = true)
|-- Customer Street: string (nullable = true)
|-- Customer Zipcode: integer (nullable = true)
|-- Department Id: integer (nullable = true)
|-- Department Name: string (nullable = true)
|-- Latitude: double (nullable = true)
|-- Longitude: double (nullable = true)
|-- Market: string (nullable = true)
|-- Order City: string (nullable = true)
|-- Order Customer Id: integer (nullable = true)
|-- order date (DateOrders): string (nullable = true)
|-- Order Id: integer (nullable = true)
|-- Order Item Cardprod Id: integer (nullable = true)
|-- Order Item Discount: double (nullable = true)
|-- Order Item Discount Rate: double (nullable = true)
|-- Order Item Id: integer (nullable = true)
|-- Order Item Product Price: double (nullable = true)
|-- Order Item Profit Ratio: double (nullable = true)
|-- Order Item Quantity: integer (nullable = true)
|-- Sales: double (nullable = true)
|-- Order Item Total: double (nullable = true)
|-- Order Profit Per Order: double (nullable = true)
|-- Order Region: string (nullable = true)
|-- Order State: string (nullable = true)
|-- Order Status: string (nullable = true)
|-- Order Zipcode: integer (nullable = true)
|-- Product Card Id: integer (nullable = true)
|-- Product Category Id: integer (nullable = true)
|-- Product Description: string (nullable = true)
|-- Product Image: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Product Price: double (nullable = true)
|-- Product Status: integer (nullable = true)
|-- shipping date (DateOrders): string (nullable = true)
|-- Shipping Mode: string (nullable = true)
```

The schema has 53 columns. Most are integer, double or string. The two date columns, `order_date` (DateOrders) and `shipping_date` (DateOrders), are read as string, so they must be converted before any date analysis (done in step 7).

2. Row count and sample data

```
total = df.count()
print("total rows:", total)

print("Sample of 5 rows:")
df.show(5)
```

```
total rows: 180519
Sample of 5 rows:
```

Category Id	Category Name	Customer City	Customer Country	Customer Email	Customer Fname	Customer Id	Customer Lname	Customer Phone Number	Customer State	Customer Street	Customer Zipcode	Department Id	Department Name	Latitude	Longitude	Order Customer Id	order date (DateOrders)	Order Id	Order Item Cardprod Id	Order Item Description	Order Item Product Price	Order Item Profit Ratio	Order Item Quantity	Sales	Order Item Total	Order State	Order Status	Order Zipcode	Product Card Id	Product Category Id	Product Description	Product Price	Product Status	shipping date (DateOrders)	Shipping Mode
1	Electronics	San Francisco	USA	john.doe@gmail.com	John	1001	Doe	415.555.1234	CA	1600 Amphitheatre	94043	1	Electronics	37.7749	-122.4312	1001	2023-10-27	1001	1	Smartphone X	999.99	0.2	1	999.99	CA	Shipped	94043	1	1	Smartphone X	999.99	Active	2023-10-27	Standard	
2	Books	New York	USA	jane.smith@yahoo.com	Jane	1002	Smith	212.555.5678	NY	200 W 4th St	10013	2	Books	40.7128	-74.0060	1002	2023-10-28	1002	2	Book Y	19.99	0.1	2	39.98	NY	Shipped	10013	2	2	Book Y	19.99	Active	2023-10-28	Standard	
3	Home Goods	Los Angeles	USA	mike.jones@outlook.com	Mike	1003	Jones	310.555.9012	CA	123 Main St	90001	3	Home Goods	34.0522	-118.2437	1003	2023-10-29	1003	3	Furniture Z	499.99	0.3	1	499.99	CA	Shipped	90001	3	3	Furniture Z	499.99	Active	2023-10-29	Standard	
4	Food & Beverage	Chicago	USA	emma.wilson@icloud.com	Emma	1004	Wilson	773.555.3456	IL	456 N Dearborn	60610	4	Food & Beverage	41.8819	-87.6308	1004	2023-10-30	1004	4	Snack A	9.99	0.05	10	99.90	IL	Shipped	60610	4	4	Snack A	9.99	Active	2023-10-30	Standard	
5	Travel	London	UK	alex.brown@bt.com	Alex	1005	Brown	203.555.7890	England	789 Baker St	W1T 3JF	5	Travel	51.5074	-0.1278	1005	2023-10-31	1005	5	Ticket B	29.99	0.15	2	59.98	England	Shipped	W1T 3JF	5	5	Ticket B	29.99	Active	2023-10-31	Standard	

```

| DEBIT| 3| 4| 91.25| 314
73|Sporting Goods| Caguas| Puerto Rico| XXXXXXXXX| Cally| 20
sumer| PR|5365 Noble Nectar...| 725| 2| Fitness| 18.
Indonesia| 20755| 1/31/2018 22:56| 77202| 1360|
17| 327.75| 0.289999992| 1|327.75| 314.64001
cidental| COMPLETE| NULL| 1360| 73| NUL
5| 0| 2/3/2018 22:56|Standard Class|
|TRANSFER| 5| 4| -249.0899963| 311
73|Sporting Goods| Caguas| Puerto Rico| XXXXXXXXX| Irene| 19
sumer| PR| 2679 Rustic Loop| 725| 2| Fitness|18.2
India| 19492| 1/13/2018 12:27| 75939| 1360|
54| 327.75| -0.800000012| 1|327.75| 311.35998
Rajastn| PENDING| NULL| 1360| 73| NUL
5| 0| 1/18/2018 12:27|Standard Class|
| CASH| 4| 4| -247.7799988| 309
73|Sporting Goods| San Jose| EE. UU.| XXXXXXXXX| Gillian| 19
sumer| CA|8510 Round Bear Gate| 95125| 2| Fitness|37.2
India| 19491| 1/13/2018 12:06| 75938| 1360|
53| 327.75| -0.800000012| 1|327.75| 309.72000
Rajastn| CLOSED| NULL| 1360| 73| NUL
5| 0| 1/17/2018 12:06|Standard Class|
| DEBIT| 3| 4| 22.86000061| 304
73|Sporting Goods| Los Angeles| EE. UU.| XXXXXXXXX| Tana| 19
ffice| CA| 3200 Amber Bend| 90027| 2| Fitness|34.1
Australia| 19490| 1/13/2018 11:45| 75937| 1360|
52| 327.75| 0.079999998| 1|327.75| 304.80999
eensland| COMPLETE| NULL| 1360| 73| NUL
5| 0| 1/16/2018 11:45|Standard Class|
| PAYMENT| 2| 4| 134.2100067|

```

The file has 180,519 rows. With 53 columns, each row wraps over several lines in the terminal. The first image shows the total and the column headers, and the second shows the values, such as payment types (DEBIT, TRANSFER, CASH, PAYMENT) and delivery statuses. The sensitive columns Customer Email and Customer Password are already masked as XXXXXXXXX.

3. Null values per column

`F.when(F.col(c).isNull(), c)` returns the column name only when the value is NULL. `F.count()` ignores rows where when returns nothing, so it counts exactly the NULLs. The list comprehension repeats this for all 53 columns in one pass.

```

print("Checking for null values across all columns...")
df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).show()

```

```
Checking for null values across all columns...
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Type|Days for shipping (real)|Days for shipment (scheduled)|Benefit per order|Sales per customer
Id|Category Name|Customer City|Customer Country|Customer Email|Customer Fname|Customer Id|Customer
Customer State|Customer Street|Customer Zipcode|Department Id|Department Name|Latitude|Longitude|N
er Id|order date (DateOrders)|Order Id|Order Item Cardprod Id|Order Item Discount|Order Item Disc
rice|Order Item Profit Ratio|Order Item Quantity|Sales|Order Item Total|Order Profit Per Order|Or
pcode|Product Card Id|Product Category Id|Product Description|Product Image|Product Name|Product P
rs)|Shipping Mode|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|  0|                0|                0|                0|                0|                0|                0|
  0|                0|                0|                3|                0|                0|                0|
    0|                0|                0|                0|                0|                0|                0|
    0|                0|                0|                0|                0|                0|                0|
55679|                0|                0|                0|                0|                0|                0|
    0|                0|                0|                0|                0|                0|                0|
```

Only four columns contain NULLs: Customer Lname (8), Customer Zipcode (3), Order Zipcode (155,679, about 86% of rows) and Product Description (180,519, so it is empty in every row). All other columns have none. The output wraps because of the number of columns, so the values appear in the same order as the headers above them.

4. Delivery status breakdown

```
print("Delivery status breakdown...")
df.groupBy("Delivery Status").count().orderBy(F.desc("count")).show()
```

```
Delivery status breakdown...
+-----+-----+
| Delivery Status|count|
+-----+-----+
| Late delivery|98977|
| Advance shipping|41592|
| Shipping on time|32196|
| Shipping canceled| 7754|
+-----+-----+
```


More than half of the orders are late: 98,977 orders (54.8%) have the status `Late delivery`. The other statuses are `Advance shipping` (41,592), `Shipping on time` (32,196) and `Shipping canceled` (7,754), and the four add up to 180,519.

5. Late delivery rate by order region

`Late_delivery_risk` is a 0/1 flag, so its average per region equals the share of late orders. The script also counts the orders per region, so the size of each region is visible.

```
print("Late delivery rate by order region...")
df.groupby("Order Region").agg(
    F.avg("Late_delivery_risk").alias("late_delivery_rate"),
    F.count("*").alias("num_orders")
).orderBy(F.desc("late_delivery_rate")).show(20, truncate=False)
```

Late delivery rate by order region...

Order Region	late_delivery_rate	num_orders
Central Africa	0.5796064400715564	1677
South Asia	0.5626697710516104	7731
East Africa	0.5593952483801296	1852
Western Europe	0.5584861116234461	27109
South of USA	0.557725587144623	4045
Eastern Europe	0.5566326530612244	3920
East of USA	0.5566160520607375	6915
Southeast Asia	0.5552992976202956	9539
Central Asia	0.5533453887884268	553
West Asia	0.5528374105508405	6009
US Center	0.5524036011550875	5887
Central America	0.5475459581525	28341
North Africa	0.5451732673267327	3232
Southern Europe	0.5438447672569187	9431
Eastern Asia	0.5432692307692307	7280
South America	0.5430867090726481	14935
Northern Europe	0.5404411764705882	9792
Oceania	0.5402049664958612	10148
West of USA	0.5395971475040661	7993
Southern Africa	0.533275713050994	1157

only showing top 20 rows

Among the 20 regions shown, the late rate runs from 53.3% (Southern Africa) to 58.0% (Central Africa). The spread is only about five percentage points, so late delivery is a problem in every region and not in one place. Central Africa has just 1,677 orders, so its rate is less reliable than that of Western Europe (27,109 orders, 55.8%).

6. Top 10 categories by total sales

```
print("Top 10 categories by total sales...")
df.groupby("Category Name").agg(
    F.sum("Sales").alias("total_sales"),
    F.sum("Order Item Quantity").alias("total_quantity")
).orderBy(F.desc("total_sales")).show(10, truncate=False)
```

Top 10 categories by total sales...

Category Name	total_sales	total_quantity
Fishing	6929653.690574931	17325
Cleats	4431942.783185065	73734
Camping & Hiking	4118425.571018985	13729
Cardio Equipment	3694843.1966299606	37587
Women's Apparel	3147800.0	62956
Water Sports	3113844.684857893	15540
Men's Footwear	2891757.6623528693	22246
Indoor/Outdoor Games	2888993.9135054224	57803
Shop By Sport	1309522.0414546824	32726
Computers	663000.0	442

only showing top 10 rows

Fishing earns the most (about 6.93 million) from only 17,325 units. Cleats sells the most units (73,734) for 4.43 million, so a Fishing unit costs far more than a Cleats unit. Camping & Hiking is third with 4.12 million.

7. Monthly sales trend

The order date is stored as text such as 1/31/2018 22:56. `to_timestamp` converts it using the pattern `M/d/yyyy H:mm`, and `date_format` turns it into a year-month label. The script then sums Sales for each month.

```

print("Monthly sales trend...")
df_time = df.withColumn(
    "order_month",
    F.date_format(F.to_timestamp(F.col("order date (DateOrders)"), "M/d/yyyy H:mm"),
"yyyy-MM")
)
df_time.groupBy("order_month").agg(F.sum("Sales").alias("monthly_sales")) \
.orderBy("order_month").show(50, truncate=False)

```

```

Monthly sales trend...
+-----+-----+
|order_month|monthly_sales|
+-----+-----+
|2015-01    |1051590.0801973708|
|2015-02    |927009.8981680475 |
|2015-03    |1051253.6903342803|
|2015-04    |1014463.2798225896|
|2015-05    |1050478.4401524863|
|2015-06    |1024006.1698320399|
|2015-07    |1038081.1900958794|
|2015-08    |1029494.6906387694|
|2015-09    |1018338.5999745401|
|2015-10    |1049154.2704332976|
|2015-11    |1029120.2406187421|
|2015-12    |1057840.8806148078|
|2016-01    |1046308.2505002206|
|2016-02    |968542.8490254203 |
|2016-03    |1025853.1208179824|
|2016-04    |1001211.5797783697|
|2016-05    |1029400.200723331 |
|2016-06    |1003059.5500050072|
|2016-07    |1045715.6208620066|
|2016-08    |1048200.2511136188|
|2016-09    |1002397.0399380219|
|2016-10    |1048130.5504065108|
|2016-11    |1047590.1410746184|
|2016-12    |1037408.1701250104|
|2017-01    |1029698.0201477306|
|2017-02    |992534.89879036   |
|2017-03    |1048004.7810653495|
|2017-04    |1038321.6208700588|
|2017-05    |1105485.321896099 |
|2017-06    |1032086.4903108279|
|2017-07    |1104373.3609792592|
|2017-08    |1109337.1712568188|
|2017-09    |1143775.1076231992|
|2017-10    |1073994.1690205582|
|2017-11    |626914.3836670096 |
|2017-12    |503910.8173430199 |
|2018-01    |331650.11515684036|
+-----+-----+

```

From January 2015 to October 2017, monthly sales stay between about 0.93 and 1.14 million and peak in September 2017 (1,143,775). February is the lowest month of each year, as in 2015 (927,010). Sales then fall to 626,914 in November 2017, 503,911 in December 2017 and 331,650 in January 2018. This drop probably reflects incomplete data at the end of the file and not a real decline, but the script cannot confirm that.

Comparison and conclusion

The three datasets have very different quality problems, so each script focuses on the checks that matter for its data.

	Healthcare	E-commerce	Supply Chain
Format	Avro	Parquet	CSV
Rows	55,500	about 107.9 million	180,519
Columns	15	9	53
Main quality findings	534 duplicate rows, negative billing amounts, mixed-case names	32% unknown category codes, 14% unknown brands, about 10% estimated duplicates	4 columns with NULLs, one column empty in every row
Analysis	Eight SQL queries on a saved Parquet table	Funnel, revenue, cart abandonment, activity by hour	Late delivery, category sales, monthly trend

Key takeaways

- **Format matters for loading.** Avro needs the external `spark-avro` jar. Parquet carries its data types, but the nanosecond `event_time` needed a special setting. CSV needs `inferSchema`, and its date columns still arrive as text.
- **Missing data hides in different forms.** The Healthcare data has no NULLs, but its text values are inconsistent. The E-commerce data replaces missing values with the word Unknown. The Supply Chain data has real NULLs, including a column that is empty in every row.
- **Healthcare is the only script that saves a table.** It turns the Avro file into a Parquet table, so analysts can query it with SQL. The other two scripts print their results.